

ENTERPRISE APPLICATIONS / ORACLE INTEGRATION

timecard submission api

HMC Sanaad B2E Platform

Request examples, Oracle procedure contract, entry validation and response mapping.
One submission. One Oracle call. 28–31 entry objects.



QATAR / EMPLOYEE SELF-SERVICE

DOCUMENT DATE

21 September 2026

VERSION

1.0 / proposal

ENTRY LIMIT

28–31 inclusive

Proposed contract — not a deployed implementation.

TECHNICAL READING GUIDE

the proposal at a glance

For the mobile, backend and Oracle teams implementing monthly timecard submission in HMC Sanaad.

DBA confirmation required. The existing project does not contain a confirmed timecard submission contract. This document preserves the proposed design and explicitly separates examples from a working database implementation.

28–31

Allowed entry-object count

31

Objects in the full example

01

Atomic Oracle submission call

01	request contract	03
02	complete request example	04
03	oracle procedure contract	06
04	oracle call & transaction	07
05	entry validation	08
06	response contract	10

Key distinction: a date-range entry counts once, regardless of the number of days covered.

Next dependency: the approved timecard persistence/workflow routine and Oracle version.

Scope: documentation only. No application or database changes are included.

SECTION 01 / 6

request contract

We propose one authenticated timecard submission endpoint. The API sends the complete entries array to Oracle in one procedure call, with validation at both boundaries.

Proposal status: No timecard submission procedure was found in the reviewed Oracle registry or compiled contracts. The endpoint and procedure in this document are proposed additions. They are not a confirmed existing or deployed implementation.

endpoint

HTTP / PROPOSED CONTRACT

```
POST /api/v1/timecards/submit
Authorization: Bearer <access_token>
Content-Type: application/json
```

header fields

Field	Proposed interpretation
p_user_name	Login username. Must match the authenticated user.
p_period	English month and year, for example July 2026.
p_confirmation_flag	Confirmation of submission; example uses Y.
p_language	Language token; example uses EN. Final supported values require agreement.
p_employee_notes	Employee's submission notes.
p_entries	JSON array containing 28 to 31 entry objects, inclusive.

authenticated identity

Retain p_user_name if required by the mobile contract, but reject a value that differs from the authenticated user. Bind the verified JWT username to Oracle, not an arbitrary body value. Existing project submit adapters already use the authenticated username.

counting rule

The limit applies to the number of objects in p_entries, not the number of calendar days covered. A two-day range counts as one object. The original three-entry example fails the new minimum. Array length does not establish complete calendar coverage; do not add dummy entries to meet the minimum.

SECTION 02 / 6

complete request example

The following body contains **31 entry objects**: 29 hourly entries and two single-day date-range entries. It preserves the hourly, cross-department and date-range shapes from the supplied request. No entries are omitted from the example.

The time values and identifiers are illustrative, not verified payroll data. In this proposed contract, p_dept_id, p_cost_center and p_comments may be omitted when not applicable.

JSON / ILLUSTRATIVE EXAMPLE

```
{
  "p_user_name": "V-AMARRI",
  "p_period": "July 2026",
  "p_confirmation_flag": "Y",
  "p_language": "EN",
  "p_employee_notes": "All OT approved by department head via email.",
  "p_entries": [
    { "p_hour_type_id": "OT_REG", "p_entry_date": "01-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "02-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "03-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "04-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "05-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "06-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "07-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "08-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "09-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "Y",
      "p_dept_id": "4471", "p_cost_center": "CC-4471",
      "p_comments": "Corporate event support" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "10-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "11-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "12-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "13-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "14-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "15-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "16-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "17-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "18-Jul-2026",
      "p_value": "2", "p_cross_dept_flag": "N" },
    { "p_hour_type_id": "OT_REG", "p_entry_date": "19-Jul-2026",
```

JSON / ILLUSTRATIVE EXAMPLE / CONTINUED

```

    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "20-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "64", "p_from_date": "21-Jul-2026",
    "p_to_date": "21-Jul-2026", "p_days": "1", "p_comments": "" },
  { "p_hour_type_id": "64", "p_from_date": "22-Jul-2026",
    "p_to_date": "22-Jul-2026", "p_days": "1", "p_comments": "" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "23-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "24-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "25-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "26-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "27-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "28-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "29-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "30-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" },
  { "p_hour_type_id": "OT_REG", "p_entry_date": "31-Jul-2026",
    "p_value": "2", "p_cross_dept_flag": "N" }
]
}

```

Count interpretation: 21 July and 22 July are separate single-day entries, each with p_days = "1". This example contains one entry for each of July's 31 days. Whether the business requires one row per day is a separate decision from the requested array-size rule.

SECTION 03 / 6

oracle procedure contract

Use a CLOB containing JSON for p_entries. The mobile API receives a normal JSON array and serializes that array only at the Oracle boundary.

Proposed procedure: XXHMC_SND_TIMECARD_SUBMIT_PR.

The following is a signature declaration for DBA agreement, **not a complete executable procedure**. The real timecard persistence and workflow contract is still required.

SQL / PROPOSED CONTRACT

```
PROCEDURE XXHMC_SND_TIMECARD_SUBMIT_PR (
  p_user_name      IN  VARCHAR2,
  p_period         IN  VARCHAR2,
  p_confirmation_flag IN  VARCHAR2,
  p_language       IN  VARCHAR2,
  p_employee_notes IN  VARCHAR2,
  p_entries        IN  CLOB,

  p_success_flag   OUT VARCHAR2,
  p_error_msg      OUT VARCHAR2,
  p_reference_no   OUT VARCHAR2,
  p_submitted_date OUT DATE,
  p_card_status    OUT VARCHAR2,
  p_approval_chain OUT SYS_REFCURSOR
);
```

approval-chain cursor

The proposed p_approval_chain cursor returns these columns, ordered by step sequence:

Column	Example
STEPSEQ	1
ROLE	LINE_MANAGER
APPROVERNAME	Fatima Al-Sulaiti
STATUS	CURRENT

Oracle must generate the real reference number, submission date, card status and approval chain. They must not be hardcoded from the sample response.

procedure responsibilities

1. Validate the header and every entry before writing.
2. Resolve the employee and validate hour-type, department and cost-center values.
3. Save the header and entries through approved Oracle timecard tables or a package.
4. Submit the timecard to the real approval workflow.
5. Return the resulting reference and approval chain.

The missing information is the approved timecard persistence/workflow contract. Guessed HR tables or invented workflow routines are not a working integration.

SECTION 04 / 6

oracle call & transaction

After the Oracle team implements the exact agreed signature, the API can execute this parameterized PL/SQL block. Procedure/schema names must remain code-owned, and request values must remain bound parameters.

SQL / PROPOSED CONTRACT

```
BEGIN
  XXHMC_SND_TIMECARD_SUBMIT_PR(
    p_user_name      => :p_user_name,
    p_period         => :p_period,
    p_confirmation_flag => :p_confirmation_flag,
    p_language       => :p_language,
    p_employee_notes => :p_employee_notes,
    p_entries        => :p_entries,

    p_success_flag   => :p_success_flag,
    p_error_msg      => :p_error_msg,
    p_reference_no   => :p_reference_no,
    p_submitted_date => :p_submitted_date,
    p_card_status    => :p_card_status,
    p_approval_chain => :p_approval_chain
  );
END;
```

json clob binding

The p_entries bind contains only the serialized array, not the complete HTTP request body. Proposed node-oracledb bind descriptor:

TYPESCRIPT / PROPOSED CONTRACT

```
p_entries: {
  dir: oracledb.BIND_IN,
  type: oracledb.DB_TYPE_CLOB,
  val: JSON.stringify(body.p_entries),
}
```

Use the existing OracleService.callMultiCursor() to retrieve both scalar outputs and approval-chain rows. It consumes cursors before releasing the connection. Returning an unread cursor through call() would release its connection too early.

atomic submission

Save the whole timecard atomically, not one committed entry at a time. The existing callMultiCursor() uses autoCommit: true. If the procedure catches a failure and returns p_success_flag = 'N', it must first undo any writes it made; otherwise partial data could be committed.

The procedure and any nested business routines must share an agreed transaction boundary. This proposal does not execute or commit a live Oracle submission.

SECTION 05 / 6

entry validation

api array-size validation

The project already uses class-validator and class-transformer for nested-array DTO validation. Apply the following property decorators in the proposed request DTO:

TYPESCRIPT / PROPOSED CONTRACT

```
@IsDefined()  
@isArray()  
@ArrayMinSize(28)  
@ArrayMaxSize(31)  
@ValidateNested({ each: true })  
@Type(() => TimecardEntryDto)  
p_entries!: TimecardEntryDto[];
```

TimecardEntryDto must separately validate its fields and enforce the permitted hourly or date-range shape. This property excerpt is not a complete DTO implementation.

Input	API result
Missing, null or not an array	HTTP 400
0–27 entries	HTTP 400
28–31 entries	Passes the count check; other validation still applies
32 or more entries	HTTP 400

oracle array-size validation

Oracle should repeat the check before any writes. This standalone validation block expects the array JSON in a CLOB bind. It checks array shape and count, not individual business fields.

SQL / PROPOSED CONTRACT

```
DECLARE
    l_entries JSON_ARRAY_T;
BEGIN
    IF :p_entries IS NULL THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'p_entries is required'
        );
    END IF;

    l_entries := JSON_ARRAY_T.parse(:p_entries);

    IF l_entries.get_size() NOT BETWEEN 28 AND 31 THEN
        RAISE_APPLICATION_ERROR(
            -20002,
            'p_entries must contain between 28 and 31 entries'
        );
    END IF;
END;
/
```

This block assumes Oracle 12.2 or later, which provides `JSON_ARRAY_T.parse(CLOB)` and `get_size()`. A malformed or non-array JSON input is rejected by parsing. The API must still provide its own HTTP validation/error mapping.

additional business validation

- Each entry uses either `p_entry_date + p_value` or `p_from_date + p_to_date + p_days`, not an ambiguous mixture.
- Dates must be real calendar dates within `p_period`, and date ranges must not be reversed.
- Numeric strings must contain valid permitted amounts. Do not infer payroll limits from the illustrative value 2.
- Cross-department entries require a valid department/cost-center combination.
- Hour-type IDs must be checked against Oracle's authoritative list. `OT_REG` and 64 remain supplied examples, not confirmed lookup values.
- The meaning of `p_days` must follow the approved business calendar; array length alone does not establish duration or complete monthly coverage.

SECTION 06 / 6

response contract

The endpoint should return the supplied response shape. The following values are an **illustrative success response**, not a captured Oracle result. In particular, the reference number, submission date and named approvers must come from the real workflow.

JSON / ILLUSTRATIVE EXAMPLE

```
{
  "status": "success",
  "errormessage": "Success",
  "request": {
    "employeeid": "V-AMARRI",
    "period": "July 2026"
  },
  "response": {
    "submitstatus": {
      "successflag": "S",
      "referenceno": "TC-2026-07-58214",
      "submitteddate": "03-08-2026",
      "cardstatus": "PENDING_APPROVAL",
      "approvalchain": [
        {
          "stepseq": "1",
          "role": "LINE_MANAGER",
          "approvername": "Fatima Al-Sulaiti",
          "status": "CURRENT"
        },
        {
          "stepseq": "2",
          "role": "SECTION_HEAD",
          "approvername": "Dr. Khalid Rahman",
          "status": "WAITING"
        }
      ]
    }
  }
}
```

output mapping

The paths under submitstatus below are relative to response.submitstatus.

Response field	Source / transformation
status	p_success_flag = 'S' maps to success; otherwise error
errormessage	Success on success; sanitized Oracle message on failure
request.employeeid	Verified login username, preserving the requested response spelling
request.period	Validated period
submitstatus.successflag	p_success_flag
submitstatus.referenceno	p_reference_no
submitstatus.submitteddate	p_submitted_date formatted DD-MM-YYYY
submitstatus.cardstatus	p_card_status
submitstatus.approvalchain	Cursor rows mapped to stepseq, role, approvername and status, ordered by step

preserve the exact envelope

The project's default ResponseInterceptor returns a different envelope. This endpoint should build its own response and use the existing @SkipEnvelope() mechanism, rather than changing the response format of other APIs.

Use the established HTTP 200 convention for returned business submission outcomes; malformed request bodies remain HTTP 400. Define the failure response contract explicitly before implementation; do not return invented success fields on failure.